Unit- 4

Structures and Union

User Defined Data Type

- **Structure:** A collection of variables (of different data types) referenced under one name, providing a useful way to keep related information together. Example: student record is a collection of roll no, name, class, marks, grade etc.
- Keyword struct is used to construct a structure

Syntax of struct:

```
struct structureName {
  dataType member1;
  dataType member2;
  ...
};
```

Example :

```
 struct sturec
 {
  int roll_no;
  char name [20];
  float marks;
  char grade;
};
```

derived type struct sturec is defined. Now, you can create variables of this type.

Creating struct Variables

- When a struct type is declared, no storage or memory is allocated. To allocate memory of a given structure type and work with it, we need to create variables.

```
struct sturec {
  // code
};
int main() {
  struct sturec student1, student2, s[20];
  return 0;
}
```

Another way of creating a struct variable is:

```
struct sturec {
   // code
} student1, student2, s[20];
```

In both cases, student1 and student2 are struct sturec variables; s[] is a struct sturec array of size 20.

Accessing Members of a Structure

Elements of structure are referred to as:

```
student1.roll_no=10;   // name of the structure variable.element name
student1.marks=60;
student1.grade='B';
```

```c
// Create a structure called myStructure
struct myStructure {
   int myNum;
   char myLetter;
};
int main() {
   // Create a structure variable of myStructure called s1
   struct myStructure s1;
   // Assign values to members of s1
   s1.myNum = 13;
   s1.myLetter = 'B';
   printf("My number: %d\n", s1.myNum);
   printf("My letter: %c\n", s1.myLetter);
   return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
My number: 13
My letter: B


--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

```c
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30]; // String
};
int main() {
  struct myStructure s1;
  // Assign a value to the string using the strcpy function
  strcpy(s1.myString, "C programming");   //error is done using s1.myString = "C programming"
  printf("My string: %s", s1.myString);
  return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
My string: C programming
--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

Simpler syntax of structure

You can also assign values to members of a structure variable at declaration time, in a single line.
Just insert the values in a comma-separated list inside curly braces `{}`. Note that you don't have to use the `strcpy()` function for string values with this technique:

```
// Create a structure
struct myStructure {
   int myNum;
   char myLetter;
   char myString[30];
};
int main() {
   // Create a structure variable and assign values to it
   struct myStructure s1 = {13, 'B', "C Programming"};
   printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);
   return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
13 B C Programming
--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

Keyword typedef

- We use the typedef keyword to create an alias name for data types. It is commonly used with structures to simplify the syntax of declaring variables.
- For example:

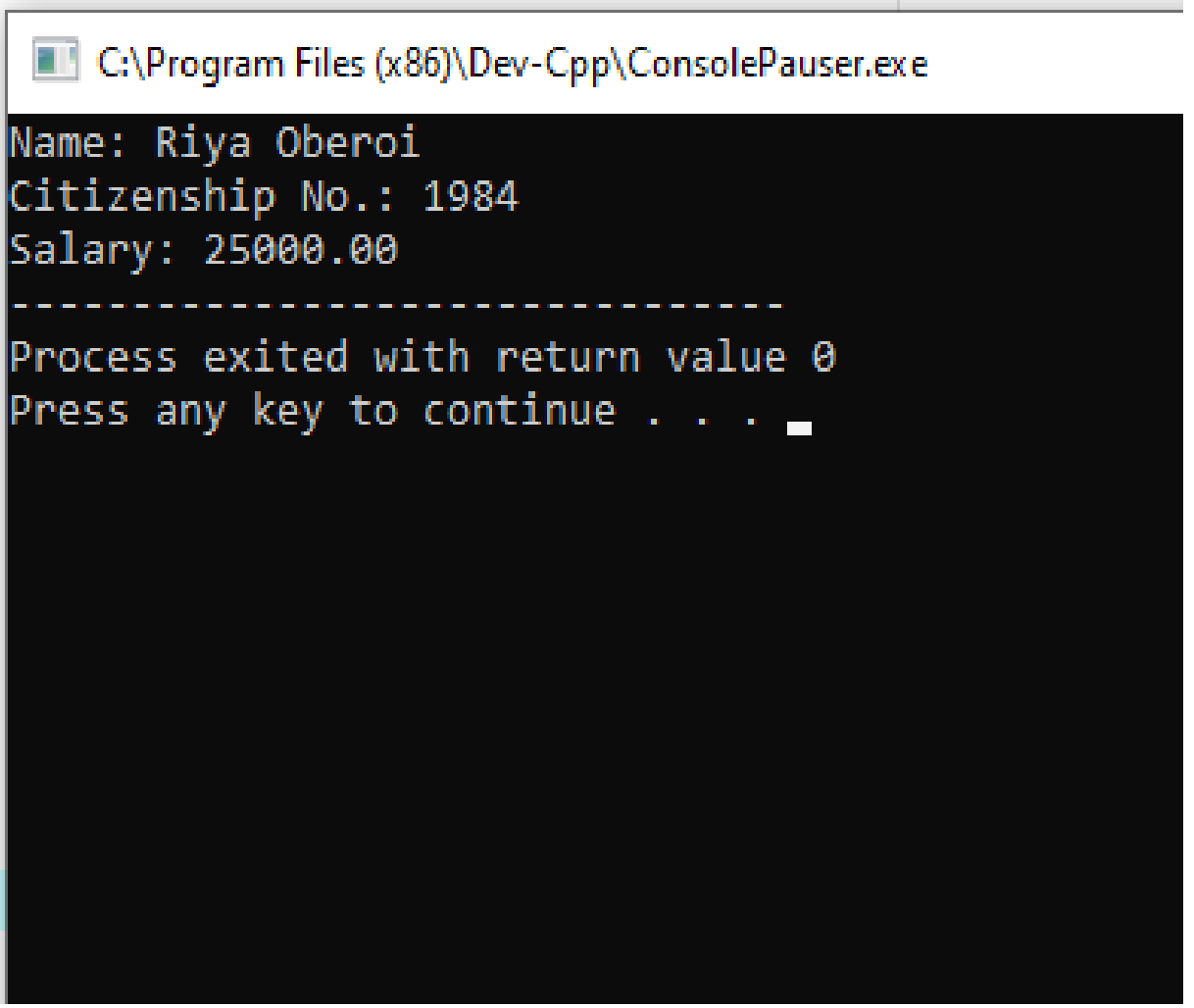
```
struct Distance{
  int feet;
  float inch;
};
int main() {
  struct Distance d1, d2;
}
```

We can use typedef to write an equivalent code with a simplified syntax:

```
typedef struct Distance {
  int feet;
  float inch;
} distances;
int main() {
  distances d1, d2;
}
```

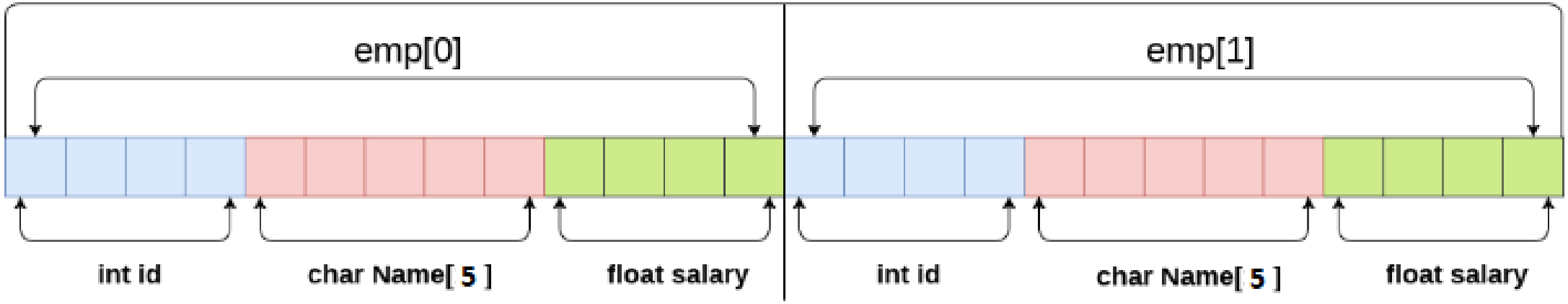
```c
#include <stdio.h>
#include <string.h>
typedef struct Person {
  char name[50];
  int citNo;
  float salary;
} person;

int main() {
  // create  Person variable
  person p1;
  // assign value to name of p1
  strcpy(p1.name, "Riya Oberoi");
  // assign values to other p1 variables
  p1.citNo = 1984;
  p1. salary = 25000;
  // print struct variables
  printf("Name: %s\n", p1.name);
  printf("Citizenship No.: %d\n", p1.citNo);
  printf("Salary: %.2f", p1.salary);
  return 0;
}
```



Array of structures

An array of structures in C can be defined as the collection of multiple structures variables where each variable contains information about different entities. The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.




```
struct employee
{
    int id;
    char name[5];
    float salary;
};
struct employee emp[2];
```

sizeof (emp) = 4 + 5 + 4 = 13 bytes

sizeof (emp[2]) = 26 bytes

Array of structures

```c
#include<stdio.h>
#include <string.h>
struct student{
int rollno;
char name[10];
};
int main(){
int i;
struct student st[3];
printf("Enter Records of 3 students");
for(i=0;i<3;i++){
printf("\nEnter Rollno:");
scanf("%d",&st[i].rollno);
printf("\nEnter Name:");
scanf("%s",&st[i].name);
}
printf("\nStudent Information List:");
for(i=0;i<3;i++){
printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
   return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Enter Records of 3 students
Enter Rollno:10

Enter Name:Riya

Enter Rollno:20

Enter Name:Arun

Enter Rollno:30

Enter Name:Vivek

Student Information List:
Rollno:10, Name:Riya
Rollno:20, Name:Arun
Rollno:30, Name:Vivek
--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

Nested Structure in C

- A nested structure in C is a structure within structure. One structure can be declared inside another structure in the same way structure members are declared inside a structure.
- Syntax:

```
struct name_1
   {
      member1;
     member2;
     .
     .
     membern;

     struct name_2
     {
        member_1;
        member_2;
        .
        .
       member_n;
     } var1;
   } var2;
```

The member of a nested structure can be accessed using the following syntax:

Variable name of Outer_Structure.Variable name of Nested_Structure.data member to access

```c
#include <stdio.h>
#include <string.h>
struct Employee    // Declaration of the dependent structure
{
  int employee_id;
  char name[20];
  int salary;
};
struct Organisation    // Declaration of the Outer structure
{ char organisation_name[20];
  char org_number[20];
  struct Employee emp;//Dependent structure is used as a member inside the main structure for implementing nested structure
};
int main()
{ struct Organisation org;
  printf("The size of structure organisation : %ld\n", sizeof(org));
  org.emp.employee_id = 101;
  strcpy(org.emp.name, "Robert");
  org.emp.salary = 400000;
  strcpy(org.organisation_name, "SIT");
  strcpy(org.org_number, "SIT1234");
  printf("Organisation Name : %s\n", org.organisation_name);
  printf("Organisation Number : %s\n", org.org_number);
  printf("Employee id : %d\n", org.emp.employee_id);
  printf("Employee name : %s\n", org.emp.name);
  printf("Employee Salary : %d\n",org.emp.salary);
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
The size of structure organisation : 68
Organisation Name : SIT
Organisation Number : SIT1234
Employee id : 101
Employee name : Robert
Employee Salary : 400000

--------------------------------
Process exited with return value 25
Press any key to continue . . .
```

Operations on structure variables

- In C, the only operation that can be applied to *struct* variables is assignment. Any other operation (e.g. equality check) is not allowed on *struct* variables.

```c
#include <stdio.h>
struct Point {
  int x;
  int y;
};

int main()
{
  struct Point p1 = {10, 20};
  struct Point p2 = p1; // works: contents of p1 are copied to p2
  printf(" p2.x = %d, p2.y = %d", p2.x, p2.y);
  return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
 p2.x = 10, p2.y = 20
--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

```c
#include <stdio.h>
struct Point {
   int x;
   int y;
};

int main()
{
   struct Point p1 = {10, 20};
   struct Point p2 = p1; // works: contents of p1 are copied to p2
   if (p1 == p2)  // compiler error: cannot do equality check for
                  // whole structures
   {
      printf("p1 and p2 are same ");
   }
   return 0;
}
```

Compile Log | Debug | Find Results | Close

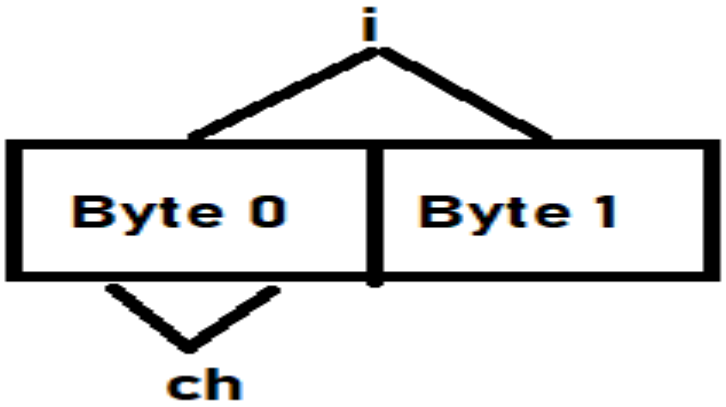
	Message
uments\struct2.c	In function 'main':
uments\struct2.c	[Error] invalid operands to binary == (have 'struct Point' and 'struct Point')

User Defined Data Type

- **Union:** A union is a special data type available in C that allows storing different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple purposes.
- When a variable is associated with a union, the compiler allocates memory by considering the size of the largest variable. So, size of union is equal to size of largest member

Example:

```
union share
 {
   int i;
   char ch;
  } data;
```



Structure vs Union

```
#include <stdio.h>
#include <string.h>
struct struct_example
{
   int integer;
   float decimal;
   char name[20];
};
union union_example
{
   int integer;
   float decimal;
   char name[20];
};
void main()
{
 // creating variable for structure, union and initializing values difference
  struct struct_example s = {18,38,"programming"};
  union union_example u = {18,38, "programming"};
  printf("structure data:\n integer: %d\n"
         "decimal: %.2f\n name: %s\n",
          s.integer, s.decimal, s.name);
   printf("\n union data:\n integer: %d\n"
           "decimal: %.2f\n name: %s\n",
          u.integer, u.decimal, u.name);
   printf("\n sizeof structure : %d\n", sizeof(s));
   printf("sizeof union : %d\n", sizeof(u));
   printf("\n Accessing all members at a time: \n");
   s.integer = 183;
   s.decimal = 90;
   strcpy(s.name, "programming");
   printf("structure data:\n integer: %d\n "
          "decimal: %.2f\n name: %s\n",
       s.integer, s.decimal, s.name);
   u.integer = 183;
   u.decimal = 90;
   strcpy(u.name, "programming");
   printf("\n union data:\n integer: %d\n "
          "decimal: %.2f\n name: %s\n",
       u.integer, u.decimal, u.name);
   printf("\n Accessing one member at time:");
   printf("\n structure data:");
   s.integer = 240;
   printf("\n integer: %d", s.integer);
   s.decimal = 120;
   printf("\n decimal: %f", s.decimal);
   strcpy(s.name, "programming");
   printf("\n name: %s\n", s.name);
   printf("\n union data:");
   u.integer = 240;
   printf("\n integer: %d", u.integer);
   u.decimal = 120;
   printf("\n decimal: %f", u.decimal);
   strcpy(u.name, "programming");
   printf("\n name: %s\n", u.name);

}
```

```
structure data:
 integer: 18
decimal: 38.00
 name: programming

 union data:
 integer: 18
decimal: 0.00
 name: ▯

 sizeof structure : 28
sizeof union : 20

 Accessing all members at a time:
structure data:
 integer: 183
 decimal: 90.00
 name: programming

 union data:
 integer: 1735357040
 decimal: 113075658868078030000000.00
 name: programming

 Accessing one member at time:
 structure data:
 integer: 240
 decimal: 120.000000
 name: programming

 union data:
 integer: 240
 decimal: 120.000000
 name: programming
```

User Defined Data Type

- **Enumeration:** It is mainly used to assign names to integral constants, the names make a program easy to read and maintain

```
enum { start,pause, go};
```

The above statement defines 3 integer constants called enumerators and assigns values to them. Enumerator values are by default assigned increasing from 0, the above declaration is equivalent to:

```
const int start=0;
const int pause=1;
const int go=2;
```

Example:

```
#include <stdio.h>
enum week { Mon,
        Tue,
        Wed,
        Thur,
        Fri,
        Sat,
        Sun };
main()
{
   enum week day;
   day = Wed;
   printf ("enum data: %d", day);
 }
```

Output:

enum data : 2